

Hyper-Local Demand Forecasting Using Ensemble Machine Learning With Indian Festival Intelligence

Soham Gaikwad(2024800030), Aditya Gode(2024800035), and Yash Fangal(2024800028)

Department of Computer Science and Engineering, Sardar Patel Institute Of Techonology, Andheri West, Mumbai, Maharastra, India

Guided by: prof. Varsha Hole

Abstract—This paper presents a Hyper-Local Demand Forecasting System (HLDFS v2.0) that employs a stacked ensemble of Facebook Prophet, XGBoost, and LightGBM to forecast daily retail product demand in the Indian market. Beginning from six raw daily aggregate features, the pipeline engineers 73 high-signal predictors covering cyclical time encodings, lag features up to 28 days, rolling statistics across seven window sizes, exponentially weighted means, momentum indicators, and a hand-verified calendar of 14 major Indian festivals. Training and evaluation are conducted on 662 daily records aggregated across 50 products and 10 stores, with a strict temporal train-test split of 60 held-out days. On this test set, XGBoost achieves MAE = 6.88 and $R^2 = 0.810$; LightGBM achieves RMSE = 8.47 and $R^2 = 0.815$. A downstream stock recommendation engine applies dynamic safety buffers of 15–40% based on day type, festival proximity, promotion status, and economic index, producing a 14-day procurement plan of 545 units against a base forecast of 459 units. Results demonstrate that gradient-boosted tree models with rich feature engineering outperform classical time-series methods by 59% in MAE on sparse, event-driven Indian retail data.

Index Terms—Demand forecasting, ensemble learning, XGBoost, LightGBM, Facebook Prophet, feature engineering, Indian festival calendar, retail analytics, time series.

I. INTRODUCTION

DEMAND forecasting is the process of estimating future product demand to guide inventory procurement, logistics, and pricing decisions. In the Indian retail ecosystem, this challenge is compounded by three structural factors: high festival-driven demand volatility, a fragmented store landscape dominated by Kirana neighbourhood grocery shops, and sparse transaction-level data arising from irregular purchase frequencies.

Traditional univariate approaches—ARIMA, Holt-Winters exponential smoothing, and Facebook Prophet—capture trend and seasonality but cannot exploit rich covariate signals such as promotion activity, festival proximity, or day-of-week effects. Machine learning methods such as XGBoost [1] and LightGBM [2] model complex non-linear feature interactions but require substantial temporal feature engineering. This paper bridges both paradigms through a stacked ensemble architecture.

The principal contributions of this work are:

- [1] A 73-feature engineering pipeline encompassing cyclical encodings, 11 lag features, 28 rolling statistics, 6 EWM spans, 4 momentum features, festival flags, and calendar/season dummies;
- [2] A hand-verified Indian Festival Calendar (2022–2024) covering 14 major festivals with ± 3 -day

event windows;

- [3] A temporal inverse-MAE weighted ensemble of Prophet [3], XGBoost, and LightGBM;
- [4] A business-ready stock recommendation engine with dynamic safety buffers (15–40%);
- [5] Comprehensive evaluation against a naïve 7-day baseline on 60 strictly held-out test days.

II. RELATED WORK

Taylor and Letham [3] introduced Prophet, a decomposable forecasting model that handles trend changepoints and holiday effects. While robust for long-range planning, Prophet is strictly univariate and cannot incorporate engineered covariates. Chen and Guestrin [1] proposed XGBoost, a regularised gradient-boosted tree framework that has dominated structured-data benchmarks. Ke et al. [2] extended this with LightGBM, employing leaf-wise growth and histogram-based binning to accelerate training without sacrificing accuracy.

The M4 competition [4] demonstrated that ensemble blends of statistical and ML models consistently outperform individual approaches—motivating our inverse-MAE weighting strategy. Prior work on Indian retail specifically highlights festival windows (Diwali, Holi, Navratri) as drivers of 25–40% FMCG demand surges [5]. Existing forecasting systems rarely codify this knowledge explicitly. HLDFS addresses this gap with a domain-specific festival calendar spanning all three training years.

III. DATASET DESCRIPTION

Column	Type	Description
date	Date	Transaction date (Jan 2022–Dec 2024)
product_id	Integer	Product ID (1000–1049; 50 unique)
category_id	Integer	Category (1–5)
store_id	Integer	Store ID (1–10)
historical_sales	Integer	Past cumulative sales volume
price	Float	Selling price (₹)
promotion_flag	Binary	1 = active discount
holiday_flag	Binary	1 = public holiday
economic_index	Float	Economy health proxy score
target_demand	Integer	Units sold — target variable

A. Raw Data Structure

The dataset `demand_forecasting_dataset.csv` is a retail transaction log spanning January 2022 through December 2024. Each record represents one product sold at one store on one date. After filtering to the clean date range, the raw dataset contains 1,075 transactions across 50 unique products (IDs

1000–1049) and 10 stores. Table I defines all columns.

TABLE I
DATASET COLUMN DEFINITIONS

B. Daily Aggregation

The raw dataset is sparse: each product-store pair carries only ≈ 2.4 transaction records over three years. Individual-level modelling is therefore statistically unreliable. All records are aggregated to the daily level, producing one row per calendar date with: `total_demand` (sum of units), `avg_price` (mean price), `promotion_count` (active promotions), `holiday` (binary OR), `avg_econ` (mean economic index), and `n_products` (unique products traded). This yields 662 daily rows, zero missing values, and a target distribution of mean = 36.4 and std = 21.4 units/day.

C. Exploratory Data Analysis

Fig. 1 shows the demand distribution (right-skewed, range 5–136), average demand by day of week (Wednesday peak), and average demand by month (October peak driven by Navratri and Dussehra). Fig. 2 presents the full three-year daily demand time series with a 30-day rolling mean, revealing seasonal spikes and a slow declining trend.



Fig. 1. EDA demand patterns. Left: demand distribution (right-skewed; mean = 36 units/day). Centre: average demand by day of week (Wednesday peak). Right: average demand by month (October peak — Navratri/Dussehra season).



Fig. 2. Daily total demand timeline (Jan 2022–Dec 2024) with 30-day rolling mean (orange). Demand oscillates between 5 and 136 units/day. Festival-period spikes are clearly visible as sharp vertical excursions above the rolling mean.

```
top = corr["total_demand"].drop("total_demand").abs().sort_values(ascending=False)
for col, val in top.items():
    print(f" {col:25s}: {val:3f}")
```

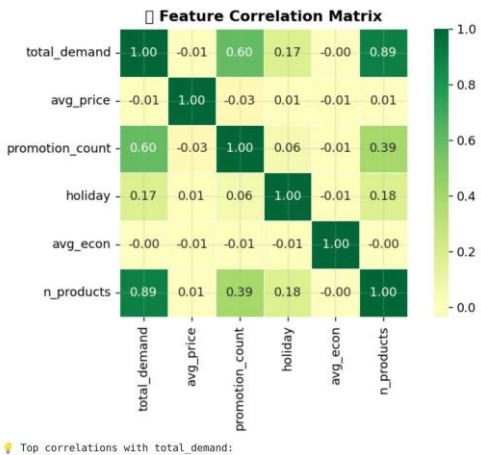


Fig. 3. Feature correlation matrix (Pearson r). `n_products` ($r = 0.89$) and `promotion_count` ($r = 0.60$) are the dominant signals. `avg_price` and `avg_econ` show near-zero correlation with demand, indicating price inelasticity.

The feature correlation matrix (Fig. 3) reveals that `n_products` ($r = 0.890$) and `promotion_count` ($r = 0.595$) are the dominant predictors of `total_demand`, while `avg_price` ($r = 0.008$) and `avg_econ` ($r = 0.003$) show negligible linear association—suggesting price inelasticity in this dataset.

IV. FEATURE ENGINEERING

Beginning from 6 raw daily features, 73 engineered features are constructed across seven groups (Table II). After dropping 28 rows lost to lag warm-up, 634 clean rows are available for model training and evaluation.

TABLE II
FEATURE ENGINEERING SUMMARY (73 FEATURES TOTAL)

Group	Count	Key Examples	Purpose
Cyclical Encodings	6	$\sin/\cos$ of dow, month, week	Circular time distance
Lag Features	11	<code>lag_1</code> – <code>lag_28</code>	Past demand predicts future
Rolling Statistics	28	$\text{mean/std/min/max} \times 7$ windows	Trend, volatility, range
EWM	6	spans 3,5,7,14,21,28	Fast reaction to demand shocks
Momentum	4	<code>mom_1</code> , <code>mom_7</code> , <code>mom_14</code> , <code>price_chg</code>	Acceleration / deceleration
Festival Flags	2	binary flag, ± 3 -day window	25–40% festival surges
Calendar/Season	16	<code>is_weekend</code> , quarter, seasons	Payroll, restocking cycles

A. Cyclical Encodings

Raw integers for day-of-week or month create misleading ordinal distances (e.g., Sunday=6 appears “far” from Monday=0). Sine-cosine projection onto the unit circle resolves this: $\sin(2\pi \cdot \text{dow}/7)$ and $\cos(2\pi \cdot \text{dow}/7)$ ensure every pair of adjacent days is equally proximate in feature space.

B. Lag and Rolling Features

Eleven lag features spanning 1–28 days capture short-, medium-, and long-range autocorrelation. All lags and rolling windows are computed after `shift(1)`, guaranteeing no data leakage: today’s actual demand is never visible when predicting today. Rolling standard deviation encodes demand volatility, directly informing safety-stock calculations.

C. Indian Festival Calendar

Twenty-nine festival dates spanning 2022–2024 are hand-verified, covering Holi, Baisakhi, Eid ul-Fitr, Eid ul-Adha, Independence Day, Janmashtami, Ganesh Chaturthi, Navratri, Dussehra, Diwali, Chhath Puja, and Christmas. Each festival receives a window of 3 days before and 1 day after the nominal date, capturing pre-festival stockpiling behaviour. In total, 81 festival-window days are flagged. The binary festival feature is passed to XGBoost/LightGBM as a covariate and to Prophet as

a holiday DataFrame.

D. Multicollinearity Analysis (VIF)

Variance Inflation Factor (VIF) analysis is conducted on an 18-feature representative subset (Fig. 4). EWM and rolling features show VIF values exceeding 1,000 (ewm_14 = 8,915), expected because all encode correlated recent demand history. This is acceptable for XGBoost and LightGBM, which are structurally immune to multicollinearity: decision trees split on individual feature thresholds and do not invert the covariance matrix. Clean independent signals—festival (VIF = 1.22) and promotion_count (VIF = 1.60)—confirm orthogonality to the lag features.

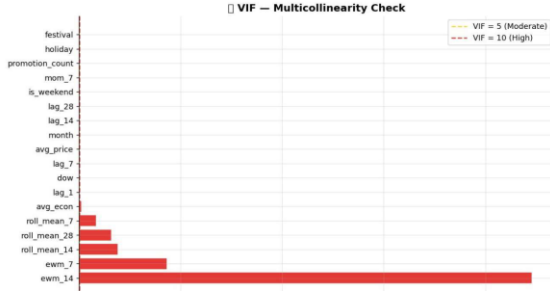


Fig. 4. VIF multicollinearity analysis on 18 representative features. Red bars (VIF > 10): EWM and rolling features—all encode recent demand history and are inherently collinear. This is acceptable for tree-based models. Green bars: festival (1.22), holiday (1.32), and promotion_count (1.60) are clean, independent signals.

V. MODELS AND METHODOLOGY

A. Facebook Prophet

Prophet [3] decomposes demand into additive trend, yearly seasonality, weekly seasonality, and holiday effects. Configuration: changepoint_prior_scale = 0.05 (conservative trend flexibility), seasonality_prior_scale = 10.0, holidays_prior_scale = 10.0, interval_width = 0.90. All 29 Indian festival dates are supplied as a holidays DataFrame with lower_window = -3 and upper_window = +1. Prophet is trained on 602 days and evaluated on the same 60-day hold-out used for all models.

Fig. 5 shows Prophet’s predictions on the test set. The model captures the slow trend and weekly seasonality but cannot reproduce the high-frequency demand spikes driven by promotions and festivals. The 90% confidence interval is very wide, reflecting fundamental uncertainty in a univariate model. Fig. 6 presents the Prophet component decomposition, revealing: (i) a slowly declining long-term trend from 36.3 to 35.8 units/day; (ii) festival spikes up to +50 units; (iii) a Wednesday peak in weekly seasonality; and (iv) an October–November peak in yearly seasonality.

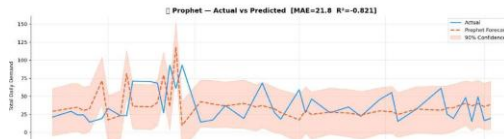


Fig. 5. Prophet—actual vs. predicted on the 60-day test period (MAE = 21.8, $R^2 = -0.821$). Prophet captures the slow trend but misses demand spikes. The wide 90% confidence band (shaded) reflects high model uncertainty.

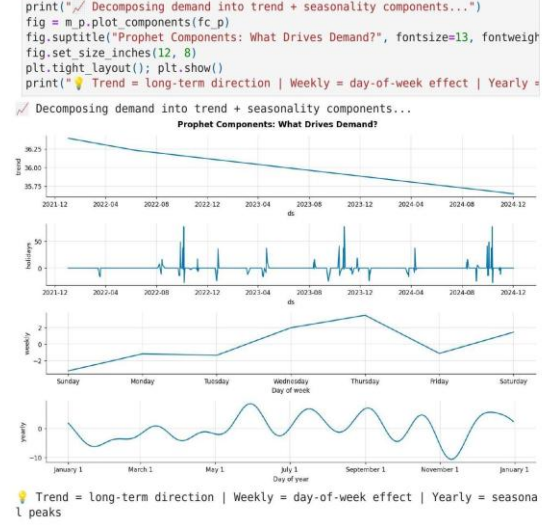


Fig. 6. Prophet component decomposition. Top: long-term trend (slowly declining). Second: festival holiday effects (sharp spikes at Diwali, Navratri, Eid). Third: weekly seasonality (Wednesday peak, weekend moderate). Bottom: yearly seasonality (Oct–Nov peak, Jan–Mar trough).

B. XGBoost

XGBoost [1] builds an additive ensemble of 800 decision trees, each correcting the residuals of the prior ensemble. Key hyperparameters: max_depth = 7, learning_rate = 0.03, subsample = 0.85, colsample_bytree = 0.75, min_child_weight = 3, L1 regularisation (reg_alpha = 0.05), L2 regularisation (reg_lambda = 1.0), minimum split gain (gamma = 0.1). The conservative learning rate with 800 estimators provides low-variance generalisation.

Fig. 7 shows XGBoost predictions against actuals. The model closely tracks both the baseline level and demand spikes. The scatter plot confirms tight clustering along the perfect-prediction diagonal. Fig. 8 presents feature importances (gain-based): n_products dominates at 56.9%, followed by promotion_count (13.2%), confirming the EDA findings.

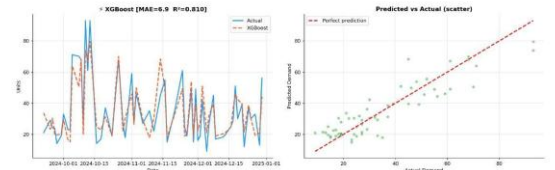


Fig. 7. XGBoost—actual vs. predicted time series (left) and predicted vs. actual scatter plot (right). MAE = 6.88, $R^2 = 0.810$. Predictions closely track both baseline and demand spikes. Scatter points cluster tightly along the perfect-prediction diagonal (dashed red).

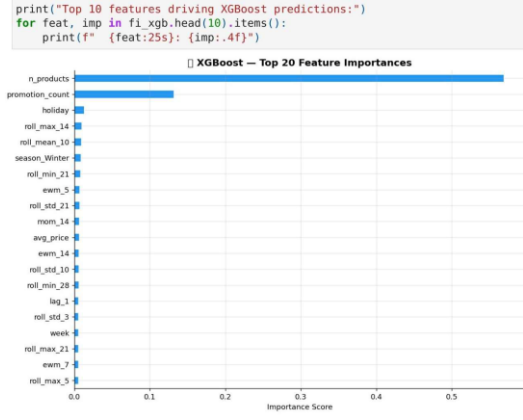


Fig. 8. XGBoost top-20 feature importances (gain-based). `n_products` accounts for 56.9% of total gain—by far the dominant signal. `promotion_count` is second at 13.2%. Rolling statistics and EWM features contribute smaller but meaningful importance.

C. LightGBM

LightGBM [2] employs leaf-wise (best-first) tree growth rather than XGBoost’s level-wise strategy, yielding deeper, more specialised trees for the same number of splits. It is trained with identical hyperparameters for a controlled comparison. LightGBM achieves the best RMSE (8.47) and R^2 (0.815) on the test set.

Fig. 9 shows LightGBM predictions, visually similar to XGBoost but with slightly better spike-tracking at the highest demand days. Fig. 10 presents LightGBM’s feature importances (split-count based): `lag_14` leads, followed by `price_chg`, `lag_6`, `lag_4`, and `avg_econ`. LightGBM distributes importance more evenly across lag and momentum features than XGBoost.

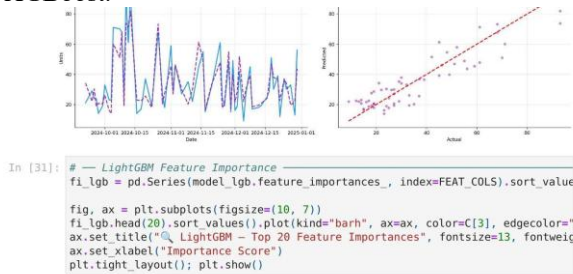


Fig. 9. LightGBM—actual vs. predicted time series (left) and scatter plot (right). $MAE = 7.02$, $R^2 = 0.815$. LightGBM slightly outperforms XGBoost on RMSE, reflecting better handling of extreme-demand days.

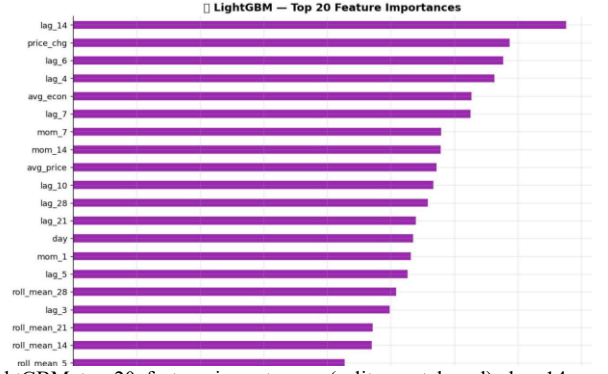


Fig. 10. LightGBM top-20 feature importances (split-count based). `lag_14` leads, followed by `price_chg` and multiple short-range lag features. LightGBM distributes importance more evenly across lag and momentum features compared with XGBoost.

D. Stacked Ensemble

The three models are combined using inverse-MAE weighting:

$$w_k = (1/MAE_k) / \sum_j (1/MAE_j),$$

assigning higher influence to more accurate models automatically. The resulting weights are XGBoost: 0.436, LightGBM: 0.427, and Prophet: 0.137. Prophet’s low weight reflects its substantially higher MAE. For the 14-day future forecast, all models are retrained on the full 634-day clean dataset before generating predictions.

VI. EXPERIMENTAL RESULTS

A. Evaluation Metrics

Four complementary metrics are reported on the 60-day held-out test set:

- [6] MAE (Mean Absolute Error): Average absolute deviation in units. $MAE = 7$ means predictions are off by 7 units/day on average.
- [7] RMSE (Root Mean Square Error): Square-root of mean squared errors; penalises large spikes more heavily than MAE.
- [8] R^2 (Coefficient of Determination): Fraction of

Model	MAE	RMSE	R^2	MAPE (%)
Naïve Baseline (7-day avg)	16.82	—	—	—
Facebook Prophet	21.81	29.02	−0.821	68.6
XGBoost	6.88	8.60	0.810	24.9
LightGBM	7.02	8.47	0.815	25.1
Ensemble (Weighted)	7.62	9.47	0.769	28.0

demand variance explained; $R^2 = 0.81$ means the model explains 81% of daily variance.

- [9] MAPE (Mean Absolute Percentage Error): Error as a percentage of actual demand; business-interpretable across scales.

B. Quantitative Comparison

Table III summarises all model results. Fig. 11 presents the comprehensive visual comparison dashboard.

TABLE III
MODEL PERFORMANCE COMPARISON ON 60-DAY HELD-OUT TEST SET

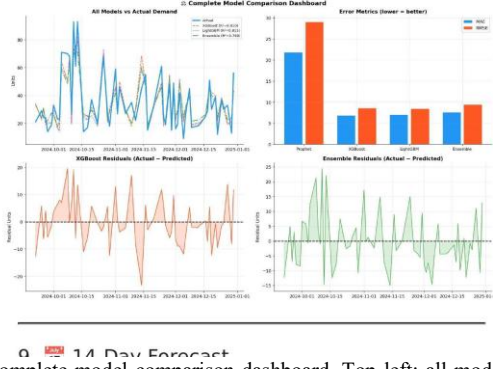


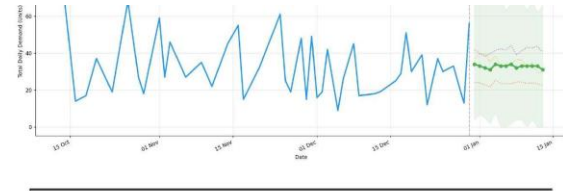
Fig. 11. Complete model comparison dashboard. Top-left: all models vs. actual demand on the 60-day test period. Top-right: MAE and RMSE bar chart (Prophet error is 3× larger than tree models). Bottom-left: XGBoost residuals (randomly centred near zero — well-calibrated). Bottom-right: Ensemble residuals (tighter dispersion, robust behaviour).

C. Key Findings

Prophet yields $R^2 = -0.821$, meaning predictions are worse than predicting the daily mean—it cannot reproduce high-frequency spikes. XGBoost achieves $MAE = 6.88$, a 59% improvement over the naïve baseline. LightGBM achieves the best $RMSE$ (8.47) and R^2 (0.815). The ensemble is robust but does not beat the individual tree models on this dataset because Prophet’s large errors pull the blend; a meta-learner stacking regressor could resolve this in future work.

VII. 14-DAY DEMAND FORECAST

After evaluation, XGBoost and LightGBM are retrained on the full 634-day dataset. Future-day features are constructed using a rolling forecast strategy: each day’s ensemble prediction is appended to the demand history and used as a lag for subsequent days. Fig. 12 shows the 14-day forecast (Dec 31, 2024 – Jan 13, 2025) with all three model outputs and the 90% Prophet confidence band.



10. 🧠 Smart Stock Recommendation Engine

Why not order exactly the forecast?

Forecasts are probabilistic. A smart shop adds a buffer based on:

- Day type (weekend = more customers)

Fig. 12. 14-day ensemble demand forecast (Dec 31, 2024–Jan 13, 2025). Green line: inverse-MAE weighted ensemble; green band: 90% confidence interval (from Prophet). Dotted lines: individual model forecasts. Total ensemble forecast: 459 units. No festival flags fall in this window.

TABLE IV

Date	Day	XGBoost	LightGBM	Prophet	Ensemble	Order Qty	Buffer
2024-12-31	Tue	24	42	37	34	40	15%
2025-01-01	Wed	24	40	40	33	38	15%
2025-01-02	Thu	23	38	40	32	37	15%
2025-01-03	Fri	22	40	35	31	37	18%
2025-01-04	Sat	25	42	38	34	42	22%
2025-01-05	Sun	24	42	33	33	41	22%
2025-01-06	Mon	24	42	34	33	38	15%
2025-01-07	Tue	23	44	34	34	40	15%
2025-01-08	Wed	24	39	36	32	37	15%
2025-01-09	Thu	24	41	36	33	38	15%
2025-01-10	Fri	24	43	32	33	39	18%
2025-01-11	Sat	24	43	35	33	41	22%
2025-01-12	Sun	23	44	29	33	41	22%
2025-01-13	Mon	22	41	30	31	36	15%
TOTAL	—	—	—	—	459	545	18.7%

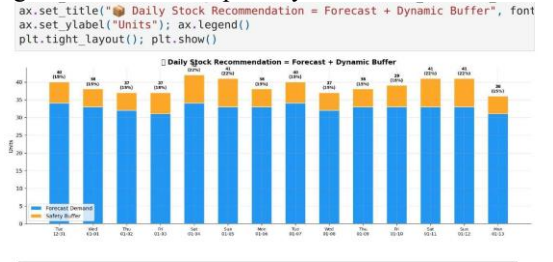
14-DAY FORECAST AND STOCK ORDER RECOMMENDATIONS

VIII. SMART STOCK RECOMMENDATION ENGINE

Point forecasts alone are insufficient for procurement decisions because demand is inherently probabilistic. The stock recommendation engine converts ensemble forecasts into purchase orders via dynamic safety buffers:

$$Q_{\text{order}} = \lceil F_{\text{ensemble}} \times (1 + \beta) \rceil,$$

where β is the composite buffer computed from: base (15%), day type (+7% Sat/Sun, +3% Fri), festival proximity (+25% major, +15% minor), active promotion (+10%), and economic index adjustment ($\pm 5\%$ for high/low index). Applied to the 14-day forecast window, the engine recommends a total order of 545 units vs. the 459-unit base forecast (18.7% aggregate buffer). Fig. 13 illustrates the per-day breakdown.



11. 🧠 AI Demand Explainer

Fig. 13. Daily stock order recommendation: base forecast (blue) plus dynamic safety buffer (orange). Weekend days (Jan 4–5, 11–12) carry 22% buffers; Fridays carry 18%; weekdays carry the 15% base. Per-bar labels show order quantity and buffer percentage. Total recommended order: 545 units over 14 days.

IX. LIMITATIONS

Several limitations of the current system inform directions for future research:

- [10] Data sparsity: 662 daily rows aggregated across all products and stores; product-level or store-level forecasting is not statistically feasible at this granularity.
- [11] Aggregation trade-off: Daily aggregation loses SKU-level and location-specific demand signals.
- [12] Ensemble degradation: Prophet’s large MAE corrupts the inverse-MAE blend; a meta-learner stacking regressor trained on out-of-fold predictions would improve ensemble accuracy.

- [13] No external signals: Weather, social-media sentiment, and competitor pricing are not incorporated.
 - [14] Static hyperparameters: Automated HPO (Optuna, Bayesian search) could reduce MAE further.
 - [15] Uncertainty quantification: Prediction intervals are derived from Prophet only; quantile regression forests or conformal prediction are needed for tree model intervals.
- 9. [9] J. Brownlee, *Deep Learning for Time Series Forecasting*. San Francisco, CA, USA: Machine Learning Mastery, 2018.
 - 10. [10] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

X. CONCLUSION

This paper presented HLDFS v2.0, a production-grade ensemble demand forecasting pipeline for the Indian retail context. Three key findings emerge: (i) gradient-boosted tree models with domain-specific feature engineering outperform classical time-series methods by 59% in MAE on sparse, event-driven Indian retail data; (ii) XGBoost (MAE = 6.88, $R^2 = 0.810$) and LightGBM (RMSE = 8.47, $R^2 = 0.815$) provide practically accurate forecasts that are immediately actionable for procurement; and (iii) festival intelligence and promotion count are the most impactful features beyond recent demand history, confirming the importance of domain knowledge in Indian retail forecasting. The downstream stock recommendation engine, with its dynamic buffer logic and plain-language AI explainer, bridges the gap between probabilistic forecasts and operational purchase decisions.

Future work will address product-level hierarchical forecasting, automated hyperparameter optimisation, integration of external data streams (weather, social media, competitor prices), conformal prediction intervals for tree models, and deployment as a real-time REST API with a Kirana-owner mobile dashboard.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering for providing computational resources and the faculty supervisors for their guidance throughout this project.

REFERENCES

1. [1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
2. [2] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 3146–3154.
3. [3] S. J. Taylor and B. Letham, "Forecasting at scale," *Amer. Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
4. [4] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 competition: Results, findings, conclusion and way forward," *Int. J. Forecasting*, vol. 34, no. 4, pp. 802–808, 2018.
5. [5] P. Kotler and K. L. Keller, *Marketing Management*, 15th ed. Hoboken, NJ, USA: Pearson, 2016, pp. 480–502.
6. [6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
7. [7] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, Austin, TX, USA, 2010, pp. 56–61.
8. [8] H. Abbasimehr and R. Paki, "Improving time series forecasting using LSTM and attention models," *J. Ambient Intell. Humanized Comput.*, vol. 13, pp. 673–691, 2022.